Introduction to Anomaly Detection

Motivation and Settings

Recent advancements in computer vision have opened up numerous opportunities to apply deep learning models in real-world applications. However, the reliability of these models remains a challenge for their practical implementation. Most machine learning models operate under the closed-world assumption, where it is presumed that the distribution of test data matches the training data. In real-world scenarios, this assumption often fails, as models may encounter outlier samples during testing that differ from the training distribution. To be reliable, a model must not only perform well in known contexts but also identify and reject anomalous inputs. Therefore, the task of "Anomaly Detection" is an important aspect of reliability and trustworthiness of the models.

There are many real-world applications that require effective anomaly detection, with industrial inspection and medical diagnosis being two of the most prominent areas. The practicality of the methods, the type of distribution shifts from normal data, and varying levels of supervision have led to the emergence of different settings and subfields within anomaly detection.

Let's align our focus by determining which of these subfields we want to work on. In both industrial and medical domains, normal and abnormal samples share the same semantics, but the abnormal samples exhibit some form of defect. For instance, in an industrial setting, the normal class might consist of images of a product, while the abnormal class would involve the exact same product but with defects such as dents or scratches. Similarly, in medical imaging, a normal brain CT-Scan might be compared to an abnormal CT-scan showing a hemorrhage. In these cases, the distribution shift between normal and abnormal data is primarily a covariate shift, as there is no semantic difference between them. For better intuition, some examples of medical (Figure 1) and industrial (Figure 2) datasets are provided below:

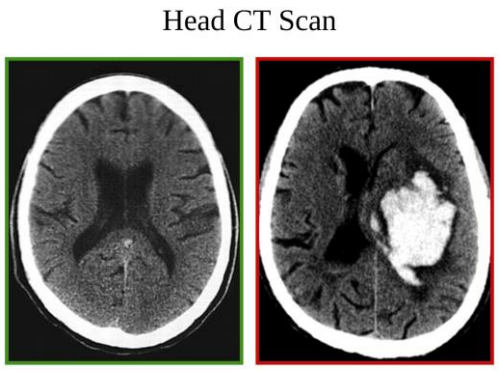

Figure 1: Head CT - hemorrhage dataset, left: normal image and right: anomalous image.

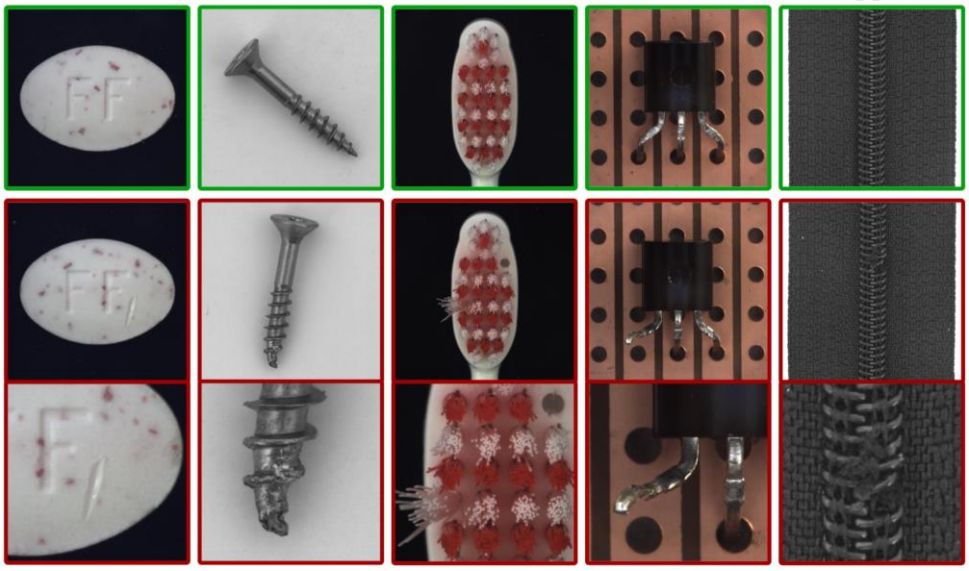

Figure 2: MVTec-AD dataset, first row: normal images and the second row: anomalous images.

Furthermore, since anomalous samples rarely appear in manufacturing product lines and also because of the unpredictable nature of anomalies, it is challenging to gather a comprehensive set of anomalous examples for training. As a result, most of the efforts in these domains are paid to unsupervised anomaly detection methods, in which we only have normal samples for the training phase.

Another aspect of the setting we are focusing on is that the model handles each class of the dataset separately. In other words, there is no need to develop a unified model that works across all classes simultaneously. This approach is commonly referred to as *one-model-per-category* or *one-class classification (OCC)* in the literature.

Output Scores

The outputs we expect from our anomaly detector are twofold:

- **Classification score**: An image-level anomaly score $s_{img} \in \mathbb{R}$, indicating how anomalous the given sample is. A higher score means the sample is more anomalous.
- **Segmentation score**: A pixel-level anomaly score $s_{pix} \in \mathbb{R}^{H \times W}$, localizing the anomalous region. Pixels with higher scores are more anomalous.

Oftentimes these scores are normalized to a real value between 0 and 1, so $s_{img} \in [0,1]$ and $s_{pix} \in [0,1]^{H \times W}$.

There is a distinction between the outputs of anomaly detection models and classification models. Here we have a real valued score that can be viewed as the certainty of the model for a sample being abnormal, rather than immediately classifying it as normal or abnormal. You can think of it as one step before

assigning a threshold and getting to a binary classification scenario. This type of output requires specific evaluation metrics, such as AUROC, AUPRO, AP, and F1 for validation.

Therefore, the problem formulation for unsupervised anomaly detection can be summarized as follows:

Given a set of anomaly-free training images, $D_{train} = \{{I_{train}}^{1}, \ldots, {I_{train}}^{n_{train}}\}$, we aim to detect if a test image from the test set, $D_{test} = \{{I_{test}}^{1}, \ldots, {I_{test}}^{n_{test}}\}$, is anomalous, and if so, to localize the anomalous regions.

Problem Statement

Zero-shot Anomaly Detection

Most current methods assume the availability of a sufficient number of normal samples during the training phase, but this assumption may not hold in all cases. For instance, in medical image processing, data confidentiality and user privacy are major concerns. Similarly, in industrial inspection scenarios, a new product line may not have enough available normal samples. These challenges highlight the need for models that can operate with little or no training data, leading to approaches like few-shot and zero-shot learning.

In this question, our focus will be on the more challenging setting of zero-shot anomaly detection. Unlike the conventional full-shot setting, which we explained earlier, these approaches involve no training data, requiring the model to achieve high performance while the only source of knowledge about the test time distribution is the implicit information in the unlabeled test set.

Being zero-shot means that the model should not see any data from the test time distribution during its training phase, neither normal nor anomalous samples. There are two accepted definitions for being zero-shot: your model can either be completely training-free, or it can have a training phase but only on an auxiliary dataset which is different from the test time distribution. For this question, we will specify one auxiliary dataset as the only allowed source for training, if you wish to have a training phase.

Your code will be tested on an industrial and medical dataset, comprising eight industrial classes and two medical classes. Considering that there are some varieties between these classes, the goal is to propose a model that generalizes well across all these dataset classes. Note that since in our setting, each class within the dataset is treated independently, the terms “dataset” and “class” are somewhat equivalent.

Therefore, the problem formulation for zero-shot unsupervised anomaly detection can be summarized as follows:

Given no training images or a set of auxiliary training images, $D_{aux} = \{{I_{aux}}^{1}, \ldots, {I_{aux}}^{n_{aux}}\}$, we aim to detect if a test image from the test set, $D_{test} = \{{I_{test}}^{1}, \ldots, {I_{test}}^{n_{test}}\}$, is anomalous, and if so, to localize the anomalous region.

Evaluation Metrics

Your output scores are evaluated by the following metrics:

Image-level evaluation metrics:

- AUROC (area under ROC curve)
- AP (average precision)
- F1

Pixel-level evaluation metrics:

- AUROC (area under ROC curve)
- AUPRO (area under PRO curve)
- AP (average precision)
- F1

Each of these metrics assesses the outcomes from a distinct viewpoint concerning precision, recall, and specificity at both the pixel and image levels. To determine your final score, which will rank you on the leaderboard, we calculate a weighted average of all the specified metrics. The weights are assigned based on the importance of each metric. This results in a final score between 0 and 100, using the following formula:

$$Final\ score\ = \frac{\sum_{metric\ \in \{AD\ metrics\}} w_{metric}\ \times metric}{\sum_{metric\ \in \{AD\ metrics\}} w_{metric}}$$

$$w_{img\_AUROC} = 1.2, \quad w_{img\_AP} = 1.1, \quad w_{img\_F1} = 1.1$$

$$w_{pix\_AUROC} = 1, \quad w_{pix\_AUPRO} = 1.4, \quad w_{pix\_AP} = 1.3, \quad w_{pix\_F1} = 1.3$$

Implementation Details

Input dataset is available at Rayan Hugging Face Page.

All the codes are available at Rayan Github Page.

Input Dataset

As mentioned earlier, your code will be evaluated on both an industrial and a medical dataset. The industrial dataset includes eight classes, and the medical dataset contains two classes. You are provided with three of the industrial classes to get a sense of what type of data your code is being evaluated on and also for your local testing. The remaining seven classes are blind and will only be accessible to your model upon submission.

When you submit your code, a directory named “data,” containing all ten classes, will be copied to the root directory of your code, and it has the following directory structure:

```
data
├── pill
│   ├── ground_truth
│   │   └── broken
│   └── test
│       ├── broken
│       └── good
├── capsules
│   ├── ground_truth
│   │   └── broken
│   └── test
│       ├── broken
│       └── good
├── photovoltaic_module
│   ├── ground_truth
│   │   └── broken
│   └── test
│       ├── broken
│       └── good
│
…
```

If you choose to include an auxiliary training phase, you are allowed to use the entire MVTec-AD dataset as your auxiliary training set. All the images (both actual images and their corresponding ground truth masks) in the dataset are square PNG files with a resolution of 518×518.

Note that

- The object name for each class (e.g., pill, capsules, photovoltaic module, etc.) is given as the name of the corresponding directory.

- There is a separate holdout dataset that differs from the ten classes previously mentioned. It can be utilized as a tiebreaker if we believe the submissions are overly fitted to the existing classes.

Provided code: We provide a Python file named "datasets/rayan_dataset.py," which contains a class called "RayanDataset" designed to be used with your data loader. This class is provided for your convenience, but if you'd like to customize your data loader, you can either create a new Dataset class or extend this one by creating a child class and using that with your data loader.

Do not alter this file as it's part of the logic used for loading and evaluating your output scores and upon submission, it will be overwritten by the original code.

Main Logic and Output Scores

In this section, you are expected to implement the full logic of your method. You should use the provided dataset and data loader, apply your method, and save the final anomaly scores for each input image.

Specifically, for every "data/…/xyz.png" file in the input dataset, you must generate both an image-level score and a pixel-level score, saving them in a JSON file named "output_scores/…/xyz_scores.json". The output of your code should produce a directory called "output_scores" with a directory structure identical to the input dataset, but each PNG file will be replaced by a corresponding JSON file.

Each JSON file should have the following format:

```
{
    "img_level_score": <scalar>
    "pixel_level_score": <2d np array>
}
```

All generated anomaly scores should be normalized to a value between 0 and 1. For pixel-level anomaly scores, outputs must have a resolution of 224×224.

Note that

- Examining the generated scores from your model can be an effective way to assess its performance and grasp why it behaves differently across various metrics. It's also a good idea to visualize the pixel-level scores and compare them with the original images.
- For security reasons, your code will be executed inside a Docker container that is isolated and does not have internet access. Therefore, if you're using any backbones or pre-trained models, you must include them in your code repository and load them offline.

- In the second phase, our automated judge will only run your code for inference. If your solution involves a training phase, save your checkpoints and just use them during inference. But keep in mind that in the final phase, the top teams will be required to submit their training code as well. We will verify whether the results are reproducible and if their training phase violates any of our mentioned rules, their results will be invalidated.

Provided code: We provide a Python file named "utils/dump_scores.py," which contains all the functions needed to save your predicted scores as described. You are free to use this file as needed.

Evaluation

Upon submission, your output scores will be evaluated using our code, and all seven of the specified metrics will be reported.

Note that

- Each submission will be run on a NVIDIA GeForce RTX 4090 GPU. Therefore, your method's inference phase must be feasible on this device, with a maximum time limit of 3 hours (including evaluation time).

Provided code: We provide a Python file named "runner.py" and a directory named "evaluation/" which contains all the necessary code for calculating the evaluation metrics. You can run and evaluate your code simply by running the "runner.py" file. This is the exact process your output scores will undergo during evaluation.

Do not alter any of these files, as they're part of the logic used for loading and evaluating your output scores, and upon submission, they will be overwritten by the original code.

Test Environment and Submission Process

Your code will run in an isolated Docker container without internet access. To help you replicate our test environment locally, we provide a sample “Dockerfile” and “docker-compose.yml.” We highly recommend running your code in this setup before submitting to avoid any unexpected dependency-related errors.

Since each submission may have unique Python dependencies, you should include a “requirements.txt” file listing your pip dependencies. Common libraries are already installed on our judge’s base image, so specifying a custom apt dependency list is unnecessary. However, if you have specific apt dependencies, you can submit a ticket, and the judging team will manually install them for you. Additionally, your code should include a shell script named “run.sh” that, when executed, runs your method and saves the output scores in the “output_scores” directory.

For submission, upload your entire repository (as specified earlier) to the Hugging Face Hub and provide us with the repository URL along with a read access token.

In summary, at test time, our files will be copied (overwritten) to the root directory of your code, and your entire repository will be structured as follows:

```
root_directory
|
├── [all your codes]
|
├── data
|   ├── pill
|   |   ├── ground_truth
|   |   |   └── broken
|   |   └── test
|   |       ├── broken
|   |       └── good
|   ├── capsules
|   |   ├── ground_truth
|   |   |   └── broken
|   |   └── test
|   |       ├── broken
|   |       └── good
|   ├── photovoltaic_module
|   |   ├── ground_truth
|   |   |   └── broken
|   |   └── test
|   |       ├── broken
```

```
│   │       └── good
│   │
│   …
│
├── datasets
│   └── rayan_dataset.py
│
├── runner.py
│
└── evaluation
    ├── base_eval.py
    ├── class_name_mapping.json
    ├── eval_main.py
    ├── json_score.py
    └── utils
        ├── json_helpers.py
        └── metrics.py
```

Additionally, your repository must include the following files with these exact names:

```
root_directory
│
├── [all the logic of your code]
│
├── requirements.txt
└── run.sh
```

References for Further Study

[1] Miyai, A., Yang, J., Zhang, J., Ming, Y., Lin, Y., Yu, Q., ... & Aizawa, K. (2024). Generalized out-of-distribution detection and beyond in vision language model era: A survey. *arXiv preprint arXiv:2407.21794*.

[2] Pang, G., Shen, C., Cao, L., & Hengel, A. V. D. (2021). Deep learning for anomaly detection: A review. *ACM computing surveys (CSUR)*, *54*(2), 1-38.